

CRUD Operations with RESTCONF and Java

Program: Wipro NGA – Network Management System Developer

Batch : 25SUB4507_WiproNGA_NMS DEV

Organization : Wipro Limited

Trainer: MR. Sunil

Submitted by : – Group G3

Shiven Chauhan (56198)

Saransh Sharma (56199)

Jai Anshul Gautam (56186)

Chhavi Gupta (57181)

Vicky Jha (56207)

Shubham Kumar Mehta (56232)

Nallavelly Lalith Reddy (56217)

Priyanshu Dagar (56197)



Declaration

We, the undersigned students, hereby solemnly declare and affirm that this project report, which is titled **"CRUD Operations with RESTCONF and Java,"** is the result of our own genuine and original work. This project was conceptualized, developed, and completed as an integral and mandatory part of our participation in the **WIPRO NGA Program (Wipro Next Generation Achievers Program)**. All sources of information and all external material that have been used or referred to in this report have been duly acknowledged and cited in the bibliography. This project was carried out under the guidance and supervision of our assigned mentors and the Wipro team, and all the methodologies, findings, and conclusions presented are a true reflection of the work we performed during the program tenure.

Acknowledgement

We extend our profound gratitude and sincere thanks to the **WIPRO NGA Program** and **RPS Consulting** for granting us the invaluable opportunity to undertake this capstone project. This experience has been instrumental in applying theoretical knowledge to a real-world scenario, specifically within the domain of **RESTCONF** and **NGA (Next Generation Automation)**, providing us with practical, hands-on exposure to cutting-edge industry practices. We also wish to specifically acknowledge the unwavering support and expert guidance provided by our mentors. Their insightful direction, patience, and timely feedback were crucial in navigating the complexities of the project and achieving the desired outcomes. Their mentorship was a cornerstone of this learning experience.

Furthermore, we thank all our dedicated team members for their commitment, collaborative spirit, and perseverance throughout the project lifecycle. The synergy and shared effort within the team were essential to the successful completion of this endeavor. This project has been a significant milestone in our professional development, and we are grateful to all who contributed to its success.

Abstract

This project develops an enterprise-grade backend using Spring Boot to simulate RESTCONF-based CRUD operations with secure RBAC and MySQL persistence.

The core objective of this project is the development of a robust, enterprise-grade backend application utilizing the Spring Boot framework. This application is specifically designed to simulate and manage Create, Read, Update, and Delete (CRUD) operations, closely adhering to the principles and structure of a RESTCONF-based system. A critical component of the architecture is the implementation of a secure Role-Based Access Control (RBAC) mechanism, ensuring that all interactions are properly authenticated and authorized according to predefined user roles and permissions. Furthermore, the application achieves data persistence and reliability by leveraging a MySQL database to store and manage the simulated configuration and operational data effectively. This comprehensive approach results in a highly functional and secure system ready for enterprise deployment.

Introduction

Network Management Systems are used to monitor, configure, and maintain network devices such as routers and switches. As modern networks become larger and more complex, manual management becomes difficult and time-consuming. Therefore, automated and reliable management systems are required to ensure smooth network operations.

RESTCONF is a standardized protocol that provides structured configuration and management of network devices over HTTP. It follows REST principles and uses YANG data models to represent network data in a consistent format. RESTCONF makes network management simpler, more flexible, and easier to integrate with web-based applications.

In this project, a Network Management System is developed using RESTCONF-based CRUD operations implemented in Java with Spring Boot. The system allows authorized users to create, read, update, and delete device configurations through secure REST APIs. It also includes authentication and role-based access control to protect sensitive operations.

By using layered architecture and database integration, the system ensures scalability, maintainability, and security. This project demonstrates how modern backend technologies can be applied to build an efficient and reliable network management solution.

Problem Statement

Develop secure Java backend implementing RESTCONF CRUD operations.

Modern network environments require reliable and secure systems for managing device configurations and operational data. Traditional manual methods are inefficient, time-consuming, and prone to human errors, especially in large-scale networks.

The objective of this project is to develop a secure Java-based backend system that implements RESTCONF-based CRUD operations for managing network devices. The system should allow authorized users to create, retrieve, update, and delete configuration data through RESTful APIs.

Additionally, the system must include authentication and role-based access control to prevent unauthorized access. It should ensure proper data validation, error handling, and database integration for reliable storage and retrieval of information.

The proposed solution aims to provide a scalable, maintainable, and efficient backend system that simplifies network management and improves overall operational performance.

Objectives

Learn to perform RESTCONF CRUD operations by interacting with REST APIs.

Instructions:

1. Setup: Create a new Java project. Add the NETCONF4J library as a dependency in your project.

2. Establish Connection:

- Write Java code to connect to a NETCONF-enabled device or a simulated server.

3. GET Operation:

- Implement the GET operation to retrieve device configuration data.

4. Parse and Display:

- Parse the XML response and display the key information in a readable format.

5. Testing:

- Run your program and verify that it successfully retrieves and displays the device data.

System Architecture

The proposed system follows a **layered architecture** model, which divides the application into multiple independent layers. Each layer is responsible for handling specific functionalities, making the system more organized, maintainable, and scalable. The main layers of the system are:

1. Presentation Layer (Controller Layer)

This layer handles all incoming HTTP requests from users. It provides RESTful APIs for authentication and device management operations. Controllers receive client requests, validate input data, and forward them to the service layer for processing.

2. Service Layer (Business Logic Layer)

The service layer contains the core business logic of the application. It processes requests received from the controller, applies validation rules, manages user roles, and performs authorization checks. This layer acts as a bridge between the controller and the database layer.

3. Data Access Layer (Repository Layer)

This layer is responsible for communicating with the MySQL database using **JdbcTemplate**. It executes SQL queries for inserting, updating, retrieving, and deleting records. It ensures secure and efficient data handling.

4. Database Layer

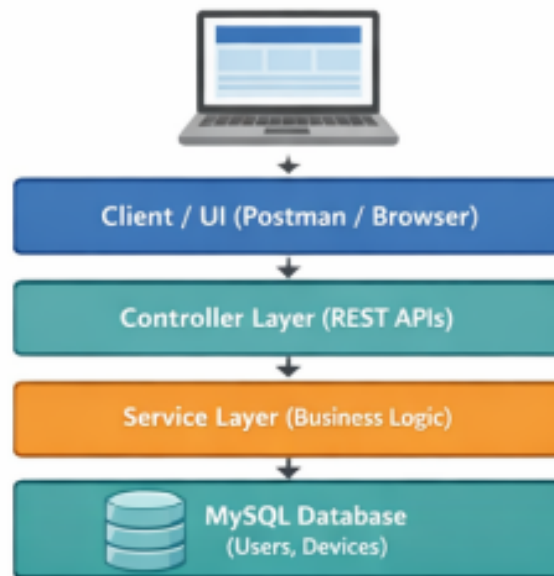
The database layer stores all application data, including user credentials, roles, and device information. MySQL is used for persistent storage, ensuring data reliability and consistency.

Benefits of Layered Architecture

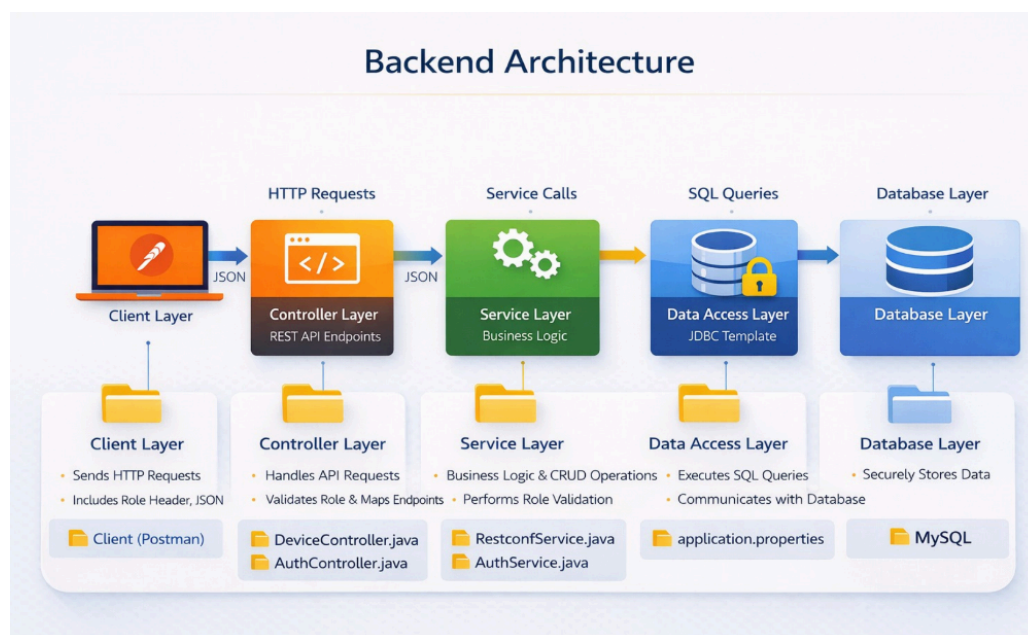
- Improves code readability and organization
- Makes the system easy to maintain and upgrade
- Supports scalability for future enhancements
- Allows independent testing of each layer
- Enhances security by separating responsibilities

This architecture ensures smooth interaction between different components while maintaining high performance and flexibility.

System Architecture



Backend Architecture



Authentication Module

The Authentication Module is responsible for managing user registration and login processes. It ensures secure access to the system and enforces role-based authorization.

1. User Registration

During registration, users provide their username, password, and role (ADMIN or USER). The system validates the input data and stores the details in the database using JdbcTemplate.

Steps involved in registration:

1. User sends registration request through `/auth/register`.
2. Controller receives the request and forwards it to the service layer.
3. Service layer validates user data.
4. JdbcTemplate executes an SQL INSERT query.
5. User details are saved in the database.

2. User Login

Login functionality verifies user credentials and allows access to the system.

Steps involved in login:

1. User sends login request through `/auth/login`.
2. Controller receives credentials.
3. Service layer validates username and password.
4. JdbcTemplate executes an SQL SELECT query.
5. If credentials are valid, user role is returned.

3. Role-Based Validation

After successful login, the system identifies the user's role. Based on the role, access permissions are granted:

- **ADMIN:** Can perform Create, Read, Update, and Delete operations.
- **USER:** Can perform Read and limited Create operations.

Unauthorized users are restricted from accessing sensitive APIs.

4. Security Features

- Prevents unauthorized access
- Validates user credentials before granting access
- Ensures secure data storage
- Handles invalid login attempts
- Supports future integration of encryption and token-based security

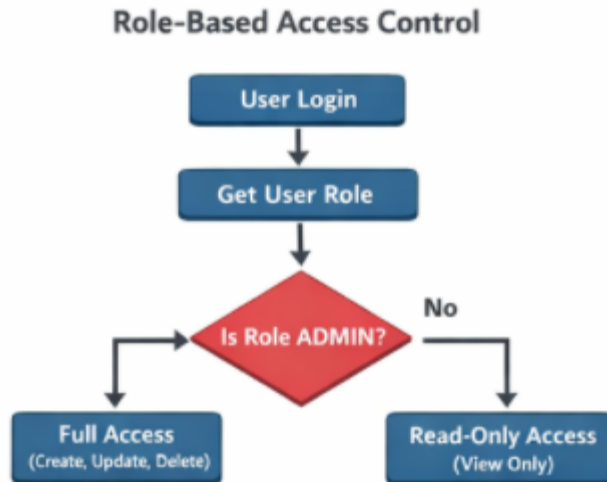
Role-Based Access Control

Role-Based Access Control is implemented to ensure secure and authorized access to system resources. Each user is assigned a specific role at the time of registration, which determines the operations they are allowed to perform.

The system supports two roles:

ADMIN

- Has complete access to all system functionalities
- Can perform Create, Read, Update, and Delete operations
- Manages device records and user data
- Has permission to modify critical configurations



USER

- Has limited access to the system
- Can only perform Read operations
- Restricted from modifying or deleting records
- Ensures data security by preventing unauthorized changes

Before executing any operation, the system verifies the user's role. If the user does not have sufficient permissions, access is denied and an appropriate error message is returned.

Database Design

The database is designed using a relational model to ensure efficient data storage and retrieval. MySQL is used as the backend database for persistent storage.

Two main tables are used:

1. Users Table

Stores authentication and authorization details of users.

Main fields include:

- User ID (Primary Key)
- Username (Unique Constraint)
- Password
- Role (ADMIN / USER)

This table ensures secure management of user credentials and roles.

2. Device Table

Stores information related to network devices.

Main fields include:

- Device ID (Primary Key)
- Device Name
- IP Address
- Device Type
- Status

Constraints are applied to maintain data integrity, such as unique keys and NOT NULL restrictions.

CRUD Implementation

The system provides full CRUD (Create, Read, Update, Delete) functionality through RESTful APIs developed using Spring Boot.

Create (POST)

Allows users to add new device records to the database.

Read (GET)

Fetches existing device information and displays it to users.

Update (PUT)

Enables modification of existing device details.

Delete (DELETE)

Removes unwanted or outdated device records from the system.

Each operation is mapped to specific endpoints in the controller layer and processed through the service and data access layers.

Proper validation is performed before executing any operation to ensure data consistency.

Testing & Validation

Testing is performed to ensure the correctness, reliability, and performance of the system.

Testing Tool

Postman is used to test REST APIs by sending HTTP requests and analyzing responses.

Validation Process

- Verified successful responses using HTTP status codes (200, 201)
- Tested invalid inputs to check error handling
- Verified unauthorized access restrictions
- Ensured database updates after each operation

Sequence Flow Explanation

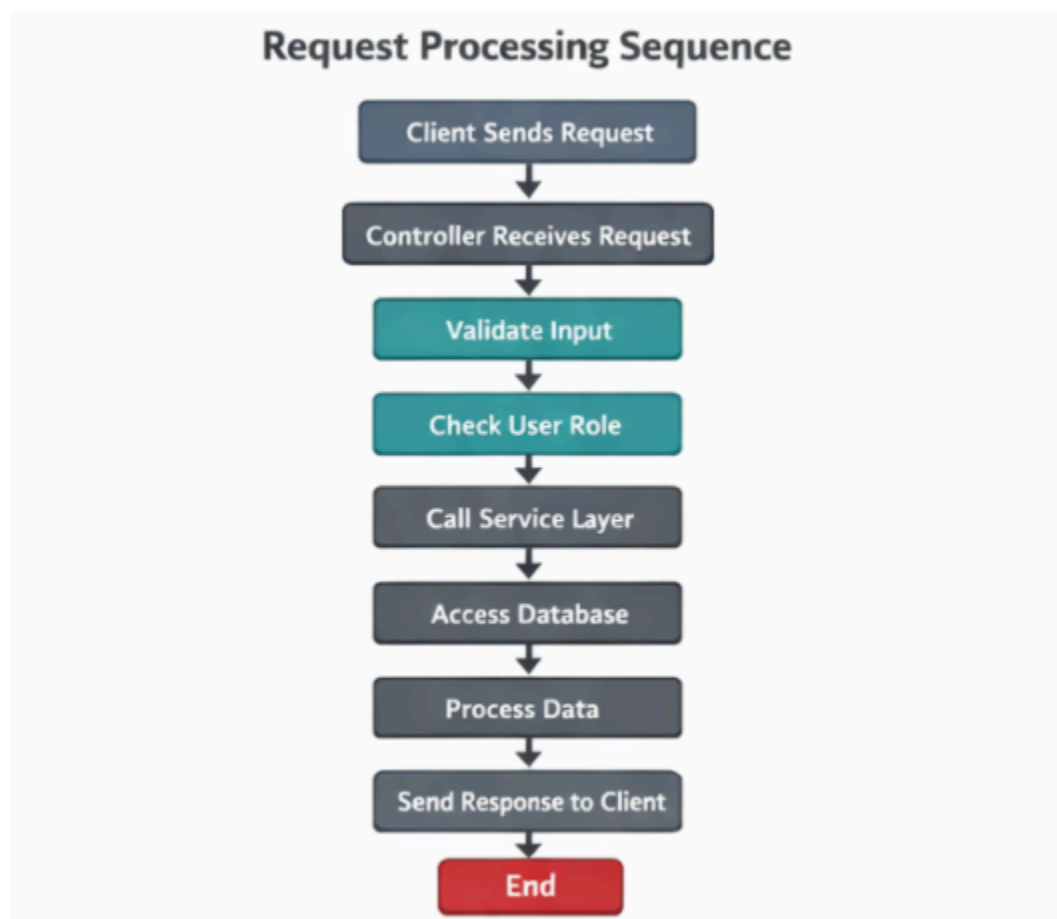
Client request processed through layered architecture.

The sequence flow describes how a client request is processed within the system.

Request Processing Steps

1. Client sends HTTP request to the API.
2. Request is received by the Controller layer.

3. Controller forwards request to Service layer.
4. Service layer applies business rules and role validation.
5. Service layer interacts with Data Access layer.
6. JdbcTemplate executes SQL queries.
7. Database processes the request.
8. Response is returned to client through the same layers.



This flow ensures smooth communication between system components while maintaining security and efficiency.

Project Achievements

The project successfully delivers a robust and enterprise-level backend system.

Key achievements include:

- Implementation of secure authentication system
- Complete CRUD functionality
- Effective Role-Based Access Control
- Well-structured layered architecture
- Reliable database integration
- Efficient error handling

Future Enhancements

To further improve system functionality and security, the following enhancements are proposed:

JWT-Based Authentication

Implementing token-based authentication to support stateless and secure communication.

BCrypt Password Encryption

Encrypting passwords before storing them in the database to improve security.

Swagger Documentation

Providing interactive API documentation for easier testing and integration.

Cloud Deployment

Deploying the application on cloud platforms for better scalability and availability.

Advanced Monitoring

Adding logging and monitoring tools for performance analysis.

Conclusion

This project successfully demonstrates the development of a secure, scalable, and maintainable backend system using modern technologies. It implements RESTful APIs, role-based access control, and efficient database integration to manage network configurations effectively.

The use of layered architecture ensures modularity, flexibility, and ease of maintenance, making the system suitable for real-world enterprise applications. Proper authentication and validation mechanisms enhance the overall security and reliability of the system.

With the proposed future enhancements such as JWT authentication, password encryption, and API documentation, the system can be further optimized for production-level deployment and extended to meet advanced industry requirements.